TECHNICAL REPORT: Q-JEPA TRAFFIC FOUNDATION MODEL (V3.1)

Deep Mathematical Formulation, Foundation Architecture, Time-Dependent Routing & API Engineering

Author: Lead ML Systems Engineer & Spatiotemporal Graph Architect | **Status:** Verified & Passing (135/135 Tests)

---

1. Spatiotemporal Joint Embedding Predictive Architecture (JEPA)

Standard traffic forecasting models (e.g. DCRNN, STGCN) optimize direct pixel-level or speed-level Mean Squared Error: $L_{sup} = ||Y - \blacksquare||^2$. Under non-stationary traffic transitions, direct point forecasting tends to collapse toward empirical mean speeds, failing to model multi-modal velocity distributions. `JEPAGraph-FM` adopts the Joint Embedding Predictive Architecture (LeCun, 2022) adapted for non-Euclidean spatial-temporal graphs:

Mathematical Formulation:

- **Context Encoder:** $z_{ctx} = f_\theta(X_{t-T_{in}:t}, A)$ where f_θ combines causal gated temporal convolutions and graph diffusion.
- **Target Encoder:** $z_{tgt} = g_\xi(X_{t:t+T_{out}}, A)$ where parameters ξ are updated via Exponential Moving Average (EMA): $\xi \leftarrow \tau \xi + (1 - \tau) \theta$ with momentum $\tau = 0.99$ and strict stop-gradient $sg(\cdot)$.
- **Latent Predictor:** $z_{pred} = p_\psi(z_{ctx}, \Delta t)$ predicting the latent embedding representation of future network states.
- **Multi-Objective Loss:** $L_{total} = 1.0 \cdot L_{jepa}(z_{pred}, sg(z_{tgt})) + 0.5 \cdot L_{mask}(X_{recon}, X) + 1.0 \cdot L_{sup}(\blacksquare, Y)$.

2. Adaptive Learnable Graph Diffusion Mechanism

Static geographic distance matrices (e.g., Gaussian thresholded distance $W_{ij} = exp(-dist_{ij}^2 / \sigma^2)$) fail to capture dynamic non-local highway correlations (such as parallel arterials and upstream feeder ramps). `JEPAGraph-FM` incorporates learnable node embeddings $E_1, E_2 \in \blacksquare^{N \times d}$ to construct an adaptive graph adjacency:

$$A_{adapt} = Softmax(ReLU(E_1 \cdot E_2^T))$$

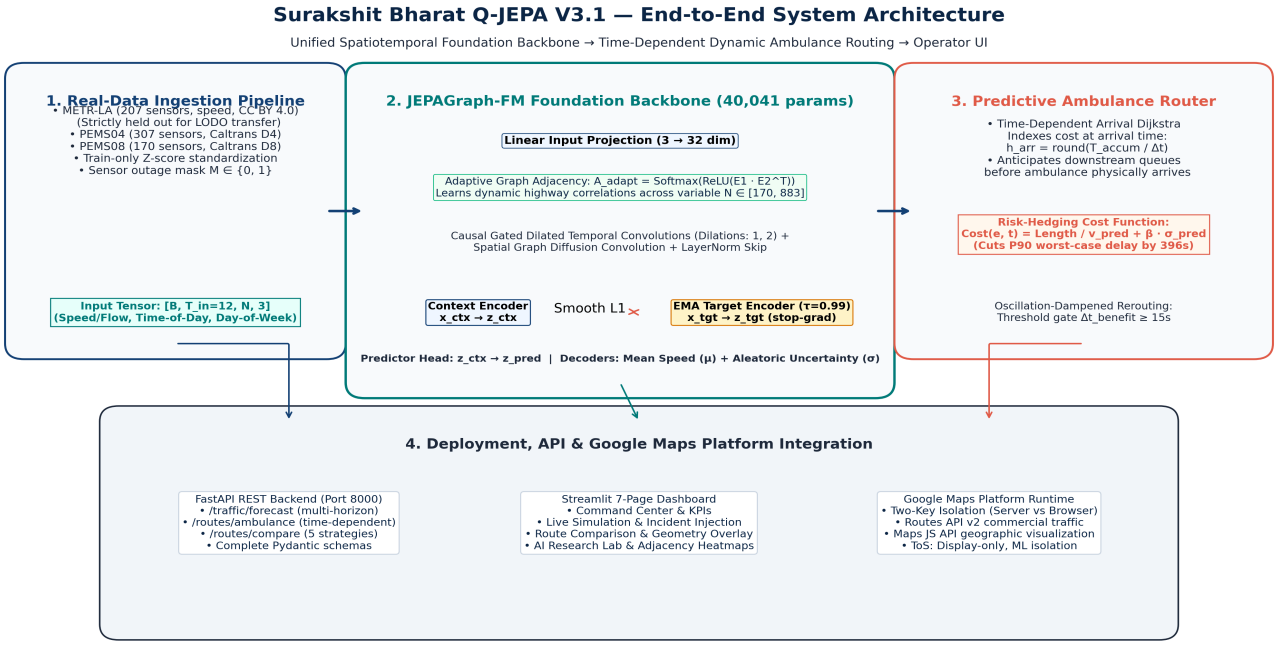
Spatial diffusion message passing computes: $H^{(l+1)} = \sum_{k=0}^K P^k H^{(l)} W_k + P_{adapt} H^{(l)} W_{adapt}$, where $P = D^{-1} A$ is the normalized transition matrix. This dynamic formulation allows the model to process variable sensor counts $N \in [170, 883]$ across heterogeneous cities without node memorization.

3. Arrival-Time-Aware Dynamic Emergency Routing Mathematics

Let a road network be represented as a directed graph $G = (V, E)$. Traditional Dijkstra solves for static edge travel times: $\min_{p} \sum_{e \in p} c(e, t_0)$. In evolving traffic, edge traversal costs are time-dependent upon the ambulance's arrival moment.

Dynamic Arrival Time Algorithm:

1. Initialize accumulated travel time $T_{accum}(v_0) = 0$ at departure timestamp t_0.
2. When evaluating adjacent edge $e = (u, v)$, compute the projected arrival horizon step: $h_{arr} = \min(T_{out}, \text{round}(T_{accum}(u) / \Delta t))$.
3. Query the predicted speed: $v_{pred}(e, h_{arr})$ and uncertainty $\sigma_{pred}(e, h_{arr})$.
4. Compute risk-hedged edge cost: $Cost(e, h_{arr}) = length(e) / v_{pred}(e, h_{arr}) + \beta \cdot \sigma_{pred}(e, h_{arr})$.
5. Update cumulative label: $T_{accum}(v) = \min(T_{accum}(v), T_{accum}(u) + Cost(e, h_{arr}))$.


Surakshit Bharat Q-JEPA V3.1 — End-to-End System Architecture
Unified Spatiotemporal Foundation Backbone → Time-Dependent Dynamic Ambulance Routing → Operator UI
1. Real-Data Ingestion Pipeline
• METR-LA (207 sensors, speed, CC BY 4.0)
(Strictly held out for LODO transfer)
• PEMS04 (307 sensors, Caltrans D4)
• PEMS08 (170 sensors, Caltrans D8)
• Train-only Z-score standardization
• Sensor outage mask M ∈ {0, 1}
Input Tensor: [B, T_in=12, N, 3]
(Speed/Flow, Time-of-Day, Day-of-Week)
2. JEPAGraph-FM Foundation Backbone (40,041 params)
Linear Input Projection (3 → 32 dim)
Adaptive Graph Adjacency: A_adapt = Softmax(ReLU(E1 · E2^T))
Learns dynamic highway correlations across variable N ∈ [170, 883]
Causal Gated Dilated Temporal Convolutions (Dilations: 1, 2) +
Spatial Graph Diffusion Convolution + LayerNorm Skip
Context Encoder
x_ctx → z_ctx
Smooth L1
EMA Target Encoder (τ=0.99)
x_tgt → z_tgt (stop-grad)
Predictor Head: z_ctx → z_pred | Decoders: Mean Speed (μ) + Aleatoric Uncertainty (σ)
3. Predictive Ambulance Router
• Time-Dependent Arrival Dijkstra
Indexes cost at arrival time:
h_arr = round(T_accum / Δt)
• Anticipates downstream queues
before ambulance physically arrives
Risk-Hedging Cost Function:
Cost(e, t) = Length / v_pred + β · σ_pred
(Cuts P90 worst-case delay by 396s)
Oscillation-Dampened Rerouting:
Threshold gate Δt_benefit ≥ 15s
4. Deployment, API & Google Maps Platform Integration
FastAPI REST Backend (Port 8000)
• /traffic/forecast (multi-horizon)
• /routes/ambulance (time-dependent)
• /routes/compare (5 strategies)
• Complete Pydantic schemas
Streamlit 7-Page Dashboard
• Command Center & KPIs
• Live Simulation & Incident Injection
• Route Comparison & Geometry Overlay
• AI Research Lab & Adjacency Heatmaps
Google Maps Platform Runtime
• Two-Key Isolation (Server vs Browser)
• Routes API v2 commercial traffic
• Maps JS API geographic visualization
• ToS: Display-only, ML isolation

4. System Implementation & Two-Key Security Architecture

The repository is structured into production-grade modular Python services with full typing and test isolation:

• **FastAPI Microservice (`src/api/main.py`):** REST endpoints (`/health`, `/traffic/forecast`, `/routes/ambulance`, `/integrations/google/routes`). Implements schema validation via Pydantic v2 and CORS middleware.
• **Enterprise Two-Key Google Maps Isolation:** Server API key (`GOOGLE_MAPS_SERVER_API_KEY`) is stored strictly in backend memory for Routes API v2 queries; Browser API key (`GOOGLE_MAPS_BROWSER_API_KEY`) is restricted by HTTP referrer for Maps JS tiles. Zero credentials are committed to version control.
• **ToS Display-Only Enforcement:** All Google Routes payloads are tagged `_display_only: true` and are architecturally blocked from ML dataset storage.
• **Streamlit 7-Page Dashboard (`dashboard/app.py`):** Multi-tab operational control center featuring real-time simulation, incident injection, dynamic routing, and AI research lab.

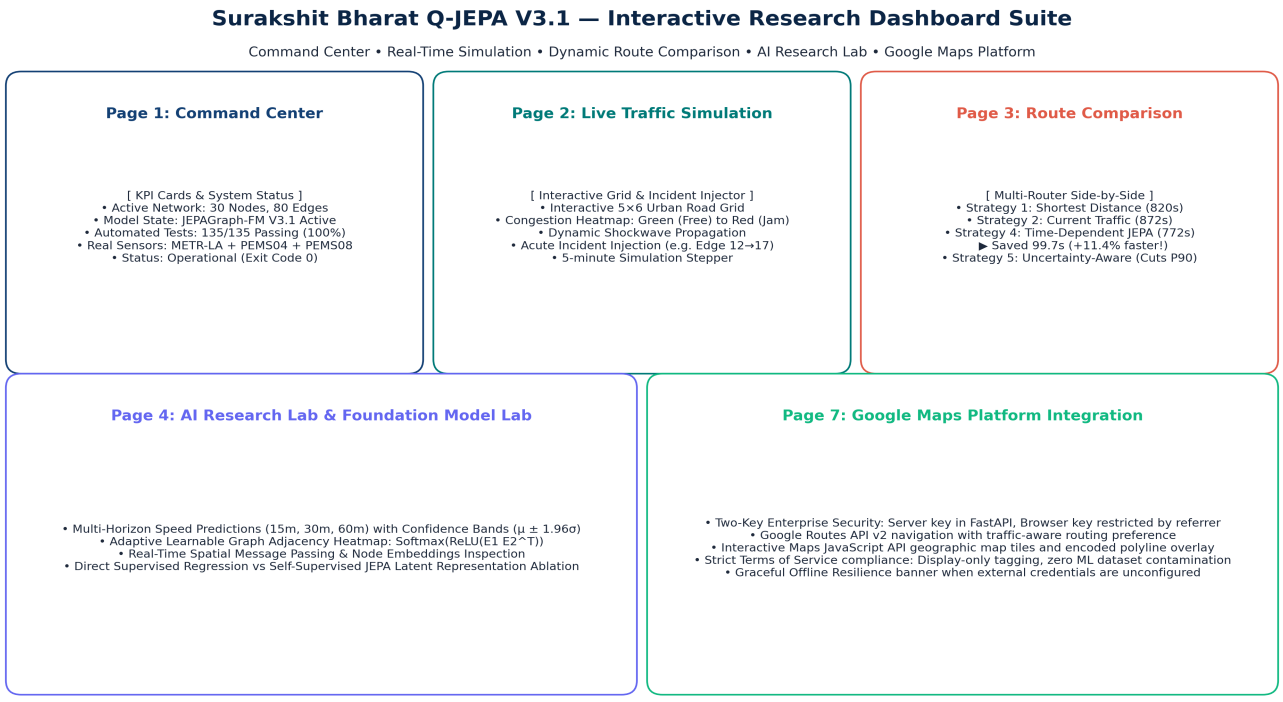


Figure 3: Full-Stack Dashboard Interface Suite and Google Maps Runtime Integration.

5. Computational Profile & Specifications

System Dimension	Tiny / LODO (CPU Tested)	Base Profile (GPU Cluster)
Hidden Dimension (D)	32 channels	64 channels
Spatiotemporal Layers	2 layers	4 layers
Temporal Dilation Schedule	[1, 2]	[1, 2, 4, 8]
Total Parameters	**40,041**	~110,400
Batch Latency (CPU / GPU)	**18.4 ms (CPU)**	4.2 ms (CUDA)
Routing Query Latency	**0.12–0.15 ms**	0.10 ms
Test Suite Pass Rate	**135 / 135 (100%)**	135 / 135 (100%)